Laya v2: team task plan (Nov 5 to Nov 18)

Nov 5, 2026 · @Abdullah Bakuvi

How to use this doc

Each teammate pastes the starter prompt below into a fresh Claude chat, attaches this doc (share the link, or export it as Markdown), and replaces NAME with their own name. Claude then works through that person's section and the team-wide section, and nothing else.

```
You are helping NAME, one of four University of Waterloo students (Murad, Abdullah, Ananya, Luke) in Velocity's Blockchain Innovation Challenge powered by Stellar (Nov 2-18, 2026). Our project is Laya v2. The attached doc is the team plan and the source of truth: read all of it before answering.

Rules for this chat:
1. Work only on NAME's section and the team-wide section. Do not do other people's tasks or edit files they own.
2. Ask me first: what is already done, what is my setup (OS, tools), and how many hours do I have today. Then give me the next task.
3. Work in small steps. Give exact commands, one at a time, and ask me to paste exact output when something fails. Explain simply and say when you are unsure.
4. Do not change the shared contract or shared TypeScript interfaces without the owner's agreement. If a change is needed, write the message I should post in the team chat.
5. Do not add features, countries, real payments, or AI-based legal judgment. Do not start a new idea.
6. Never claim anything in code, the demo or the deck that is not true (no live partners, no live funders, no legal certainty). If the doc lists something as unknown, ask me before assuming.
7. Give honest pushback. Prefer decisions and next actions over open-ended options.
```

This plan reflects the state of the project on the morning of Thursday Nov 5, 2026. Where a task says "confirm", check the repo or ask the owner before building on it.

Laya v2 in one page

Laya checks each pre-departure bill against a published rulebook, sends it to whoever legally owes it, pays the provider directly, lends the worker only what she legitimately owes, and refuses illegal fees. Every payment lands on a public receipt on Stellar (testnet only).

The problem. Maria, a Filipina domestic worker going to Hong Kong, is handed a pile of bills: training, a medical exam, a placement fee. Nobody sorts out who legally owes which. Her recruiter says she owes all of it and sends her to a partner lender charging 100% a year or more. So she borrows for costs that were never hers. Problem space: 1, Access to Fair Financial Assets and Affordable Financing.

Roles

Role	Who in the demo	What they do
Worker	Maria	Lists each bill, picks her repayment share (1% to 15%), signs the request
Provider	Training center, clinic	Must be on the verified list. Signs that the invoice amount is exact (`confirm_request`)
Employer	Hong Kong household	Pays employer-owed bills directly. Grant mode, worker owes 0
Mission funder	Bayanihan Microfinance	Pays worker-payable bills directly. Loan mode, flat fee of at most 8%
Family	Lorna (Maria's mother)	Receives the rest of each remittance. Never signs
Admin / reviewer	Our admin account	Manages the verified list, sorts "needs review" bills. Cannot move money

The rulebook is a table in the app for one route (Philippines to Hong Kong, domestic work). Each row says who owes a bill type and the fair-price cap, with a link to the official source. Laya uses no AI to judge legality.

Bill (demo)	Amount	Rule	What happens
Medical exam	65 USDC	Employer-owed	Employer pays the clinic directly. Maria owes 0

Bill (demo)	Amount	Rule	What happens
Required training	190 USDC (about 11,000 pesos)	Worker-payable	Funder pays the training center. Maria owes 190 plus 3% flat fee = 195.70
Placement fee	1,380 USDC (about 80,000 pesos)	Prohibited	Refused on screen, flagged, never becomes a request
Unknown type, price above cap, or unverified provider	any	No matching rule	Marked "needs review". The admin sorts it or refuses it

How it works, step by step

1. Maria lists each bill. The app sorts it by the rulebook, shows the result and the source link, and hashes the invoice file in her browser (the file never leaves it). She signs; that creates one request per accepted bill on Stellar.
2. The provider confirms the exact amount with its own signature. No signature, no payment.
3. The right party pays the provider directly and in full (`fund_request`). Employer: Grant. Funder: Loan. The payment and the loan record happen in one atomic transaction, so a failed payment creates no loan.
4. Maria repays from remittances she sends through Laya. One transaction splits each remittance: 15% to the funder, the rest to Lorna. No remittance means the balance does not change. After more than 3 missed periods the loan is flagged overdue (no on-chain penalty; the funder follows up off-chain). In the demo one "month" is 60 seconds.
5. A public receipt shows each bill, its rule, who paid, the provider's confirmation and the invoice fingerprint (SHA-256). It proves who agreed, who paid, how much, and that the file is unchanged. It does not prove the original invoice was genuine.

What changed from v1.1. The old lender's acceptance of a "legal amount" is replaced by the provider's confirmation of the invoiced amount. `legal_amount` is now just the principal (no 8% a year added). Employer-paid Grant mode is a first-class path, not an extra. A rulebook decides who pays. The contract keeps every function, field and error name.

Related docs: v2 brief · handoff doc (Nov 5) · `docs/SPEC.md` in the repo is still v1.1 and is superseded by this doc wherever they differ.

Dates, deadlines and checkpoints

The one date that matters most is **Friday Nov 6, 9 PM**: the full flow must run on testnet (H5). Our own feature freeze is Thu Nov 12; the organizer deadline that matters is the one-page overview on Fri Nov 13 at 8:30 AM.

Handoffs and where they stand (as of Nov 5, 10 AM)

#	Handoff	From to	Due	Status
H0	App skeleton, types, helpers, MockLayaClient	Abdullah to all	Nov 3, 11 PM	Done, pushed to `main`
H1	Demo data (providers, people, sample invoices)	Luke to Ananya	Nov 4, 6 PM	Unknown. Luke confirms
H2	Testnet accounts and test USDC	Murad to all	Nov 4, 11 PM	Unknown. Not in the repo yet (`testnet.json` still says `deployedAt: placeholder`). Murad confirms
H3	Shared UI components	Ananya to Luke	Nov 5, 6 PM	Due today
H4	Deployed contract and bindings	Murad to Abdullah	Nov 5, 11 PM	Due today. Nothing from Murad is in the repo yet
H5	Real client live, `smoke.ts` passes	Abdullah to Ananya, Luke	**Nov 6, 9 PM**	Hard checkpoint
H6	All demo steps on the real chain	Ananya, Luke to all	Nov 8	
H7	Hosted on Vercel	Abdullah to all	Nov 9	

Also done: Freighter connect with Testnet check, Horizon `getBalance` and `hasTrustline` (Abdullah), and the `translate.ts` and `errors.ts` helpers.

Calendar

Date (2026)	What	Notes
Thu Nov 5	Advisor meetings booked (deadline 5 PM ET). Working session #1, 4-6 PM, in person, whole team	Team 6 holds two slots on Nov 9 (below). Calendar invites arrive Friday
Thu Nov 5	H3 at 6 PM, H4 at 11 PM	
Fri Nov 6, 9 PM	**H5 hard checkpoint**	If missed: team meets that night, cut scope (drop `set_share` and `cancel_request` first) or demo on the mock plus one real testnet transaction
Sat Nov 7	`demo-reset.sh`, `getEvents`, `/admin`, `/family`, old-loan card	
Sun Nov 8	H6. First full run of the demo script	
Mon Nov 9	H7 Vercel. Advisor meetings, virtual: Keesan Mahalingam 1:30-2:00 PM, Abhishek Punia 4:30-5:00 PM	Abdullah attends. Ask the validation questions listed under Team-wide
Tue Nov 10	Technical workshop 5-6 PM (virtual). Deck v1	
Wed Nov 11	Working session #2, 5-6 PM, in person. Pitch prep workshop 6-7 PM. Deck v2	
Thu Nov 12	**Our feature freeze.** Bug fixes only, each with a test	
Fri Nov 13, 8:30 AM	**Deadline: upload one-page solution overview**	Likely the basis of the Top 16 cut
Fri Nov 13 to Sun Nov 15	Test on other laptops and a phone. Backup video. Two full rehearsals with judge questions	
Mon Nov 16	Project showcase, 4-7 PM, in person. Top 16 teams	Run demo-reset an hour before

Date (2026)	What	Notes
Tue Nov 17	Finalist pitch prep, 11 AM-1 PM (Top 8 only). Fixes from showcase feedback only	Tuesday night is the real deck deadline
Wed Nov 18, 10 AM	**Deadline: upload final slide deck**	Seven hours before finals
Wed Nov 18	Finals and awards, 5-7:30 PM, in person	Run demo-reset an hour before

Daily habit. A 10-minute check-in each night in the team chat: what is done, what is blocking, which handoff is next. Merge to `main` at least once a day, in small commits. Run `npm test` and `npm run build` before you push.

Team rules: ownership, shared interface, guardrails

Four people, one owner per folder, so merges rarely conflict. Pages never import `@stellar/*`; they reach Stellar only through `useLaya()` (the `LayaClient` interface) and the helpers in `app/src/lib`. Only Abdullah's real client translates between contract formats and app formats.

Repo: https://github.com/Murad-D-11/team-6-blockchain-innovation-challenge-project (public, owned by Murad). Commit small, merge to `main` daily. Secret keys live in `.env.local` and are never committed.

Owner	Files
Murad	`contracts/laya/`, `packages/laya-bindings/`, `scripts/*.sh`, `app/src/config/testnet.json` (only Murad touches it)
Abdullah	`app/src/lib/**`, `app/scripts/smoke.ts`, `app/src/app/dev/`, `app/src/app/page.tsx`, `package.json` and lockfile, `layout.tsx`, `globals.css`, app config files
Ananya	`app/src/components/**` (including all shadcn `components/ui`), `app/src/app/worker/**`, `app/src/app/family/**`

Owner	Files
Luke	`app/src/app/funder/**`, `app/src/app/creditor/**`, `app/src/app/proof/[id]/**`, `app/src/app/admin/**`, `app/src/data/**`, `app/public/samples/**`

Need a new npm package? Ask Abdullah. Need a change in someone else's file? Say so in the team chat and stop; never edit it.

Fixed stack. Next.js 16.4.4 (App Router, TypeScript, Tailwind, shadcn/ui), Vitest 5.1.0, `@stellar/stellar-sdk` 17.3.0 or later, `@stellar/freighter-api`, Rust `soroban-sdk` 30.x. Testnet only, RPC `https://soroban-testnet.stellar.org`. Nobody upgrades versions after today.

Units. All token amounts are `bigint` with 7 decimals (1 USDC = `10_000_000n`). Never use floats for money. Convert only with `toUnits` and `fromUnits` from `app/src/lib/units.ts`. Timestamps are unix seconds. Show `PHP_PER_USD = 58` as an "illustrative rate".

Testing pages on the mock. Add `?role=worker|family|funder|creditor|admin|employer` to any URL to act as that account with no Freighter. `?mockReset=1` wipes the mock's saved state. `/dev` runs the full golden flow on the mock.

Wallet. The Chrome extension Abdullah installed is branded "Varial" and appears to be Freighter renamed. Check its ID at `chrome://extensions` against the official Freighter ID `bcacfldlkkdogcmkkibnjlakofdplcbk`, and make sure all four teammates use the same wallet, set to Testnet.

Decisions made in this plan (v2.0). These settle the open design points. If you disagree, say so in the team chat today, before building on them.

1. **Eight testnet accounts**: admin, issuer, worker, family, funder, creditor (the training center), clinic, employer. `testnet.json` `accounts` gains `clinic` and `employer`. Fallback if time runs out: seven accounts, with the creditor account serving both bills.
2. **Role type** gains `"employer"`. `useWallet()` returns role `"creditor"` for both the creditor and clinic accounts; pages tell them apart by address using `providers.json`.
3. **Refusals live in the app only.** The contract never sees a prohibited bill.

4. **Bill type is not stored on-chain.** The worker picks a type in the form; each request's payer is derived from the provider's `kind` in `providers.json`. So the funder, employer, proof and admin pages need no extra data.
5. **Names do not change.** In the contract and TypeScript, `creditor` means provider and `legalAmount` means the invoiced amount. The folder `app/src/app/creditor/` stays; the page says "Provider" on screen.
6. **Funder role pays Loan only; employer role pays Grant only.**
7. **One contract change** (Murad): `legal_amount = principal`. Details and new golden numbers are in the next section.

Guardrails. No new features, countries, AI-based legal interpretation or real payments. No new ideas and no more open-ended review loops: build. Any change to the shared interface needs the owner's agreement, a post in the team chat, and a version bump in this doc. Claim only what the demo shows.

Shared contract and golden numbers for v2

Everything in `docs/SPEC.md` "Shared contract A" (the Soroban contract) and "Shared contract B" (types, `LayaClient`, translation rules) stays exactly as built, with the changes below. Where this section and SPEC.md disagree, this section wins.

Contract change (Murad), one formula. In `request_payoff`, set `legal_amount = principal` (v1.1: `principal + principal * 800 * months / 120000`).

- `months` stays in the interface and its 1 to 60 validation stays. It no longer affects any amount. In the demo use any value, for example 12.
- Everything else is unchanged: `fund_request` pays `legal_amount` to the provider and creates the loan in one transaction. Loan mode: `outstanding = legal_amount + legal_amount * fee_bps / 10000` with `fee_bps` at most 800. Grant mode: `outstanding = 0`, status Repaid. `remit`, the share floor, the pause clock (1 USDC or more resets it) and the overdue rule are unchanged.
- Keep every function, field and error name, so the bindings and client do not churn.

TypeScript changes (Abdullah). `formulas.legalAmount(principal, months)` returns `principal`. `Role` gains `"employer"`. The mock follows the same rule.

New golden numbers (7 decimals, 1 USDC = 10_000_000). Murad's Rust tests and Abdullah's mock and Vitest tests must all produce these exactly. They replace the v1.1 table.

Case	Input	Expected
Invoice amount	principal 1_900_000_000 (190 USDC), months 12	legal_amount 1_900_000_000
Loan fee	legal 1_900_000_000, fee_bps 300	outstanding 1_957_000_000 (195.70)
Normal remit	amount 4_000_000_000, share 1500	to_lender 600_000_000, to_family 3_400_000_000, outstanding 1_357_000_000 (135.70)
Final remit	outstanding 157_000_000, amount 4_000_000_000, share 1500	to_lender 157_000_000, to_family 3_843_000_000, status Repaid
Grant (medical exam)	principal 650_000_000 (65 USDC), Grant, fee 0	legal_amount 650_000_000, outstanding 0, status Repaid. Then remit 4_000_000_000 gives 0 and 4_000_000_000
Dust remit	amount 50_000_000, share 1500	to_lender 7_500_000, pause clock not reset
Share floor	`set_share(1400)` when min_share_bps is 1500	Error 7
Fund unconfirmed	`fund_request` on an Open request	Error 5
Failed funding	funder balance below legal_amount	Transaction reverts, no loan created (add a test)

Repayment path for the 190 USDC loan with 400 USDC remittances at a 15% share: 195.70, then 135.70, 75.70, 15.70, then 0 (the last remittance sends 15.70 to the funder and the rest to family).

New config shape. `app/src/config/testnet.json` `accounts` now has eight keys (Murad writes it):

```
"accounts": { "admin": "G...", "issuer": "G...", "worker": "G...", "family":
"G...", "funder": "G...", "creditor": "G...", "clinic": "G...", "employer":
"G..." }
```

Starting balances: worker 2,000, funder 10,000, employer 10,000 (test USDC). Worker, family, funder, creditor, clinic and employer all hold a trustline. Family never signs. Import worker, funder, creditor, clinic, employer and admin into Freighter on every demo laptop, set to Testnet. Register both providers with `add_creditor` at deploy time.

New data files (Luke owns the files; Abdullah owns the logic that reads them).

`app/src/data/providers.json` replaces `creditors.json`. Fictional names; rename freely.

```
[
  { "address": "G...creditor account", "name": "Pasig Skills Training
Center", "kind": "training_center", "verified": true, "regNo": "TC-0000
(fictional)", "fictional": true },
  { "address": "G...clinic account", "name": "Bayside Medical Clinic",
"kind": "clinic", "verified": true, "regNo": "MC-0000 (fictional)",
"fictional": true },
  { "address": null, "name": "Golden Gate Recruitment Agency", "kind":
"agency", "verified": true, "regNo": "EA-0000 (fictional)", "fictional": true
}
]
```

`app/src/data/rulebook.json` has one row per bill type. The caps below are illustrative demo values, not official price caps, and the UI says so.

```
[
  { "billType": "medical_exam", "providerKind": "clinic", "payer":
"employer", "capUsd": 100, "label": "Pre-entry medical exam",
    "reason": "The Hong Kong employer is responsible for the helper's pre-
entry medical exam.",
    "sourceLabel": "HK Labour Department: FDH guide", "sourceUrl":
"https://www.fdh.labour.gov.hk/res/pdf/text_alternative/FDHguideEnglish.pdf"
},
  { "billType": "training", "providerKind": "training_center", "payer":
"worker", "capUsd": 250, "label": "Required pre-departure training",
    "reason": "Philippine agencies may still charge training fees. About
11,000 pesos is a reported benchmark, not an official cap.",
    "sourceLabel": "SCMP: zero placement fee policy", "sourceUrl":
"https://amp.scmp.com/news/asia/southeast-asia/article/2176773/hong-kong-
urged-investigate-local-partners-illegal" },
  { "billType": "placement_fee", "providerKind": "agency", "payer":
"prohibited", "label": "Placement fee",
    "reason": "The Philippines has a zero placement fee policy for domestic
workers. Hong Kong caps agency commission at 10% of the first month's wages,
payable only after placement.",
    "sourceLabel": "HK Employment Agencies Administration", "sourceUrl":
"https://www.eaa.labour.gov.hk/en/helpers.html" }
]
```

`people.json` keeps its v1.1 shape, plus `"employer": { "name": "Desert Pearl Hotel (employer)", "type": "Grant" }` (a Hong Kong household name is also fine).

Classification logic (Abdullah, `app/src/lib/rulebook.ts`). Ananya's and Luke's pages all call this one function so they never disagree.

```
export type BillType = "medical_exam" | "training" | "placement_fee" |
"other";
export type ProviderKind = "clinic" | "training_center" | "agency";
export type Payer = "employer" | "worker" | "prohibited" | "needs_review";
export interface Rule { billType: BillType; providerKind: ProviderKind;
payer: "employer" | "worker" | "prohibited";
  capUsd?: number; label: string; reason: string; sourceLabel: string;
sourceUrl: string }
export interface ProviderEntry { address: string | null; name: string; kind:
ProviderKind; verified: boolean; regNo: string; fictional: true }
export interface Classification { payer: Payer; rule: Rule | null; reason:
string; blocksRequest: boolean }

export function classifyBill(input: { billType: BillType; providerKind:
ProviderKind; verified: boolean; amount: bigint }, rules: Rule[]):
Classification;
export function classifyRequest(req: { creditor: string; legalAmount: bigint
}, providers: ProviderEntry[], rules: Rule[]): Classification;
```

Rules of the function: no rule for the bill type and provider kind, an unverified provider, or an amount above `capUsd` gives `needs_review`. A `prohibited` rule gives `prohibited`. Otherwise the payer is the rule's payer. `blocksRequest` is true for `prohibited` and `needs_review`, so those bills never become on-chain requests. `classifyRequest` looks the provider up by the request's `creditor` address, takes its `kind`, and picks the bill type from the matching rule, so no bill type is stored on-chain.

Murad: contract, accounts, deploy, bindings

You own everything on-chain: the `laya` contract and its tests, the test USDC token, the testnet accounts, deployment, the TypeScript bindings, and `testnet.json`. Abdullah's real client is blocked until your deploy lands tonight (H4), so push early and post in the team chat as soon as `testnet.json` and the bindings are on `main`.

```
I'm Murad, the on-chain owner on a 4-person team building Laya v2 on Stellar
testnet. The attached team plan is the source of truth; "Shared contract and
golden numbers for v2" overrides docs/SPEC.md. Do not rename anything, change
types, add parameters or change error numbers. The only contract change is
legal_amount = principal in request_payoff. My files: contracts/laya/,
packages/laya-bindings/, scripts/*.sh, app/src/config/testnet.json. Never
edit others' files. Use soroban-sdk 30.x, #[contractevent] for events,
__constructor for setup. Every golden number and every error needs a unit
test. If anything is ambiguous, stop and ask me. First ask what is already
done and pushed.
```

Tasks

- [ ] **Now.** Post in the team chat what is done, and push everything you have (contract, tests, scripts) to `main` of the team repo. As of this morning the repo has no `contracts/`, `packages/` or `scripts/` folder, and `testnet.json` still says `deployedAt: placeholder`. Check the contract and token IDs in `testnet.json` on stellar.expert (testnet): if they do not resolve, they are placeholders.
- [ ] **Contract change.** In `request_payoff` set `legal_amount = principal`. Keep the `months` validation (1 to 60). Replace the v1.1 golden tests with the v2 table, and add a test that a funding that fails for low balance creates no loan.
- [ ] **Accounts (H2).** `scripts/setup-testnet.sh` (idempotent) creates and friendbot-funds eight identities: admin, issuer, worker, family, funder, creditor, clinic, employer. It creates trustlines to `USDC:<issuer>` for worker, family, funder, creditor, clinic and employer, deploys the asset contract, pays worker 2,000, funder 10,000 and employer 10,000, and writes accounts, asset and `tokenContractId` into `testnet.json`. Fallback if short on time: seven accounts, skipping `clinic`, and say so in the team chat.
- [ ] **Today by 11 PM, deploy (H4).** `scripts/deploy.sh` builds, deploys with constructor arguments (admin, tokenContractId, 60, 3), calls `add_creditor` for both the creditor and the clinic account, writes `contractId` and `deployedAt` into `testnet.json`, then runs `stellar contract bindings typescript` into `packages/laya-bindings`. Commit the bindings and `testnet.json`. Open the generated file and tell Abdullah exactly how enums and `RemitResult` came out.
- [ ] **Deploy order.** Make the formula change before tonight's deploy, since it is one line. If you are blocked, deploy the v1.1 contract and tell Abdullah in the team chat right away, because his smoke test numbers differ (the v1.1 numbers are in `docs/SPEC.md`). Redeploy is about 5 minutes with idempotent scripts.

- [ ] **Share keys privately.** Send each teammate the secret keys they need (worker, funder, creditor, clinic, employer, admin) through a private channel, never in a commit or the group chat, so they can import them into Freighter on Testnet. Abdullah also needs them in his `.env.local` for `smoke.ts`.
- [ ] **Fri Nov 6.** Pair with Abdullah until `smoke.ts` passes (H5, 9 PM). Debug contract errors with him.
- [ ] **Sat Nov 7.** `scripts/demo-reset.sh`: tops worker back to 2,000, funder to 10,000 and employer to 10,000, and calls `set_period(60, 3)`. Run it an hour before every demo and rehearsal.
- [ ] **Nov 8 to Nov 11.** Hardening: more tests, check TTL extension (threshold 120,960, extend to 518,400 ledgers), help Ananya and Luke debug contract errors. Write a 5-line "how the contract works" explainer for the pitch.
- [ ] **Thu Nov 12 on.** Code freeze. Only bug fixes, each with a test.

Done means: `cargo test` passes every v2 golden number and error case; the contract is live on testnet with both providers registered; `testnet.json` is complete with eight accounts; the bindings and scripts are committed.

You wait on: nobody. **Others wait on you:** H2 (accounts and token) and H4 (deployed contract and bindings).

Abdullah: client, mock, real chain, smoke test, Vercel

You own the bridge between the pages and Stellar: shared types, the mock client, the real client, wallet hooks, the rulebook logic, and the hosted app. Ananya and Luke need your v2 changes to the mock and `rulebook.ts` today, so do those first; the real client waits on Murad.

```
I'm Abdullah, the integration owner on a 4-person team building Laya v2 on
Stellar testnet. I run Windows with PowerShell and Claude Code in the
terminal. The attached team plan is the source of truth; "Shared contract and
golden numbers for v2" overrides docs/SPEC.md. My files: app/src/lib/**,
app/scripts/smoke.ts, app/src/app/dev/, app/src/app/page.tsx, package.json,
lockfile, layout.tsx, globals.css and config files. The real client is the
only place that translates contract formats to app formats. Never edit
others' files. Give step-by-step commands one at a time and ask for exact
output when something fails. First ask what is already pushed.
```

Done already: H0 (skeleton, helpers, `MockLayaClient`, `/dev`), Freighter connect with a Testnet check, Horizon `getBalance` and `hasTrustline`, and the `translate.ts` and `errors.ts` helpers. Not yet tested: the wording when the user rejects the wallet popup.

Tasks

- [ ] **Now.** Run `git pull`, then post in the team chat what you see from Murad. Export this doc as Markdown, commit it as `docs/PLAN-v2.md`, and update the "Source of truth" line in `CLAUDE.md` to point to it (and say SPEC.md is v1.1, superseded where they differ).
- [ ] **Today, first priority: v2 changes in `app/src/lib/`.** `legalAmount` returns `principal`. `Role` gains `"employer"`. Update `config.ts` types for the eight accounts. `useWallet` maps the creditor and clinic accounts to role `"creditor"`, and the employer account to `"employer"`. Replace the v1.1 golden tests with the v2 table, in both `formulas` tests and the mock tests. Add a mock test that a funding that fails for low balance creates no loan. Support `?role=employer` in the mock.
- [ ] **Today, first priority: `app/src/lib/rulebook.ts`.** Implement `classifyBill` and `classifyRequest` exactly as typed in the shared-contract section, with tests for: employer-owed medical exam, worker-payable training, prohibited placement fee, amount above cap, unverified provider, unknown type. Push and tell Ananya and Luke, since both pages depend on it.
- [ ] **Today, `/dev` flow.** Update it to the v2 demo: a 190 USDC training bill funded as Loan at 3%, and a 65 USDC medical bill funded as Grant by the employer.
- [ ] **Session 3, after Murad's H4 lands (Opus).** `RealLayaClient` against Murad's generated bindings: writes use the bindings `Client` with Freighter's `signTransaction` and `signAndSend()`, reads use `.result`. Apply the translation table exactly, and map failures to `LayaError` using the codes. Check addresses with the SDK's StrKey helpers, not a "starts with G" test.
- [ ] **Fri Nov 6, by 9 PM: H5 hard checkpoint (Session 4, Opus).** `app/scripts/smoke.ts` runs on testnet with the keys from `.env.local`: request for 190 USDC (share 1500), provider confirms, funder funds as Loan at 300 bps (outstanding 1,957,000,000), `remit` 4,000,000,000 (600,000,000 to the funder, 3,400,000,000 to family, outstanding 1,357,000,000); then a 65 USDC request funded as Grant by the employer (outstanding 0). Print all numbers. Then tell the team to set `NEXT_PUBLIC_USE_MOCK=false`. Pair with Murad.
- [ ] **Sat Nov 7.** `getEvents`: read `getHealth` for the oldest ledger, never request older, map the 5 event types, return `[]` on any failure. Pages must work with no events.

- [ ] **Sun Nov 8 (H6).** Fix integration bugs from Ananya and Luke. Ask them for the exact error text.
- [ ] **Mon Nov 9 (H7).** Deploy to Vercel (`laya-bindings` in `transpilePackages` , a `buffer` polyfill, `NEXT_PUBLIC_USE_MOCK=false`). Test on a phone and a second laptop. Advisor meetings, virtual: Keesan Mahalingam 1:30-2:00 PM and Abhishek Punia 4:30-5:00 PM (questions under Team-wide).
- [ ] **Nov 10 to Nov 11.** Loading states, and a friendly message for every `LayaError` code in `lib/errors.ts` . Test the popup-rejected wording. Confirm the wallet extension ID. Help wherever the team is behind.
- [ ] **Thu Nov 12 on.** Freeze; bug fixes only. Review Luke's one-page overview before the Nov 13 8:30 AM upload. You present the "how it works" part of the demo.

Done means: both clients implement every `LayaClient` method, the mock and the real chain give the same numbers, `smoke.ts` passes, and the app is live on Vercel.

You wait on: Murad's H2 and H4. Until then, build against the spec. **Others wait on you:** `rulebook.ts` and the mock update (today), H5 (Nov 6), H7 (Nov 9).

Ananya: shared UI, worker and family pages

You own the screens the story is told through: Maria the worker and her mother Lorna. You also own the shared components Luke reuses (H3, due today 6 PM). You build against the mock and never touch Stellar code. Demo step numbers below refer to the demo script in the Team-wide section.

```
I'm Ananya, the worker-and-family UI owner on a 4-person team building Laya
v2 on Stellar testnet. The attached team plan is the source of truth. I build
UI only: I call Stellar exclusively through useLaya() and the LayaClient
interface, and use helpers from app/src/lib (units.ts, formulas.ts, hash.ts,
rulebook.ts, useWallet). Never import @stellar packages in my pages. Never
write my own classification logic: call classifyBill from
app/src/lib/rulebook.ts. My files: app/src/components/** and
app/src/app/worker/**, app/src/app/family/**. Never edit others' files.
Stack: Next.js App Router, TypeScript, Tailwind, shadcn/ui, mobile-first.
Every write shows a loading state, then the tx link or a friendly LayaError
message. If anything is ambiguous, stop and ask me. First ask what is already
done.
```

Shared components (`app/src/components/` , H3 today at 6 PM): `PageShell` (header with role, connected address, Testnet badge), `AmountDisplay` (USD with PHP underneath using `PHP_PER_USD` , labeled illustrative), `TxLink` (stellar.expert link from `explorerTxUrl`), `RoleGuard` (shows "Switch Freighter to the Worker account" when `useWallet().role` does not match), `StatusBadge` (Open, Confirmed, Funded, Cancelled, Active, Repaid, Paused, Overdue). Also add every shadcn `components/ui` piece the team needs, and tell Luke when it is committed.

Pages

- **`/worker/request` : "list your bills".** One row per bill: bill type (medical exam, training, placement fee, other), provider (dropdown from `providers.json`), amount in USD with PHP shown, and an invoice file (hash it with `hashFile` ; the file never leaves the browser). For each row call `classifyBill` and show the result live: who owes it, the reason, and the source link. A `prohibited` bill shows a clear refusal state ("Refused: the Philippines has a zero placement fee policy", with the source link) and cannot be submitted. A `needs_review` bill shows "Sent to review" and also cannot be submitted. Below the rows: repayment share slider (1% to 15%, default 15%) and the family address (prefilled from `testnet.json`). Submit calls `requestPayoff` once per accepted bill, in list order (`months` is a fixed 12, not shown). Show each request's status afterwards, with a Cancel button while Open or Confirmed. Add a line above the signature: "Your signature is your consent to this request."
- **`/worker` dashboard.** The worker's requests and loans (`listRequests` , `listLoans` filtered by worker address). A Grant loan shows "Paid by employer: you owe 0". A Loan shows status (Active, Repaid, **Paused n of 3** when `missedPeriods > 0` , Overdue), outstanding, total repaid, total sent to family, and the share slider (cannot go below `minShareBps` , calls `setShare`).
- **Send home** (on `/worker`): amount input, a preview of the split with `formulas.split` before signing, then `remit` . Result card: "60.00 to the funder, 340.00 to Lorna", the new outstanding, the tx link.
- **"Without Laya vs with Laya" card** (on `/worker`). Static and clearly labeled "illustrative". Without Laya: the three bills total 1,635 USDC, and borrowed at 54% a year the interest alone is about 74 USDC a month. With Laya: the placement fee is refused, the employer pays the medical exam, and Maria owes 195.70 USDC, with the loan paused when no money is sent. No input fields and nothing stored.
- **`/family` .** Lorna's balance (`getBalance`), the total received from Maria (sum of `totalToFamily` over loans where family is this address), and a short explanation. No signing.

Tasks

- [ ] **Today by 6 PM, H3.** Shared components committed. Tell Luke.
- [ ] **Today.** `/worker/request` working on the mock with `?role=worker`. Use Abdullah's `classifyBill` as soon as he pushes `rulebook.ts` (he is doing it first today); until then use a clearly marked placeholder, and remove it afterwards.
- [ ] **Fri Nov 6.** `/worker` dashboard, send home, share slider and the Paused status on the mock (wait 60 seconds to see Paused).
- [ ] **Sat Nov 7.** `/family` and the "without vs with Laya" card. After H5, switch to the real chain with `NEXT_PUBLIC_USE_MOCK=false`.
- [ ] **Sun Nov 8, H6.** Run demo steps 1, 7, 8 and 9 on testnet with Freighter. Send any client bug to Abdullah with the exact error text.
- [ ] **Nov 9 to Nov 11.** Polish: mobile layout, empty states, error messages, consistent copy.
- [ ] **Thu Nov 12 on.** Freeze. You present the Maria story part of the demo.

Done means: demo steps 1, 7, 8 and 9 run on testnet without help, on a phone-sized screen.

You wait on: Abdullah's `rulebook.ts` (today) and H5 (Nov 6), and Luke's `providers.json` and `rulebook.json` (H1). Until then use the mock and placeholders.
Others wait on you: H3 (today).

Luke: data, provider, funder, proof and admin pages, deck, one-pager, outreach

You own the other side of the market (providers, funders, employers), the public receipt judges will remember, the demo data and rulebook, and the pitch. You build against the mock and never touch Stellar code. Demo step numbers refer to the demo script in the Team-wide section.

```
I'm Luke, the market-side pages and pitch owner on a 4-person team building
Laya v2 on Stellar testnet. The attached team plan is the source of truth. I
build UI and data only: I call Stellar exclusively through useLaya() and the
LayaClient interface, and use helpers from app/src/lib (including
classifyBill and classifyRequest in rulebook.ts, which I must not
reimplement). Never import @stellar packages in my pages. My files:
app/src/app/funder/**, creditor/**, proof/[id]/**, admin/**, app/src/data/**,
app/public/samples/**. Reuse Ananya's components from app/src/components once
available. Never edit others' files. Stack: Next.js App Router, TypeScript,
Tailwind, shadcn/ui. If anything is ambiguous, stop and ask me. First ask
what is already done and what has been sent.
```

Data and files (H1)

- `app/src/data/providers.json`, `app/src/data/rulebook.json`, and the `employer` entry in `people.json`, exactly as shaped in the shared-contract section. Fill the creditor and clinic addresses from `testnet.json` once Murad's H2 is in. Keep `creditors.json` until no page imports it, then delete it.
- Three fictional one-page invoices in `app/public/samples/`, each stamped "SAMPLE - FICTIONAL": a 190 USDC training invoice, a 65 USDC medical exam invoice, and a 1,380 USDC placement fee invoice.

Pages

- **`/creditor`, shown on screen as "Provider".** Requests where `creditor` equals the connected address (works for both the creditor and clinic accounts). For each Open request show the bill (from `classifyRequest`), the invoiced amount, and the text "By confirming, you state that the invoice amount is exactly this." The **Confirm invoice** button calls `confirmRequest`.
- **`/funder`, for two roles.** Role `funder` sees Confirmed requests whose payer is `worker` and can fund them as **Loan** only (fee 0% to 8%, default 3%). Role `employer` sees Confirmed requests whose payer is `employer` and gets a **Pay as employer** button (Grant, fee 0, "worker owes 0"). A funder never sees employer-owed requests and vice versa. Each request shows the worker (from `people.json`), the provider name and registration, the amount, the rule with its source link, the share, and for Loan a projected payoff estimate (outstanding divided by share times a typical 400 USDC remittance, labeled an estimate). A **Verify document** box hashes a dropped file with `hashFile` and shows a match or a mismatch against `docHash`. A **Portfolio** tab lists loans where `funder` is the connected address: outstanding, repaid, remit count, status (Active, Repaid, Paused n of 3, **Overdue** in red) and a link to the proof page.

- **`/proof/[id]`, public, no wallet.** Reads `getRequest` and `getLoan` only, never depends on events. Shows: the bill and who owed it (employer-owed or worker-payable) with the rule's source link; provider name and registration; invoiced amount; provider confirmed at (time); paid by whom and in which mode; what Maria owes now; the document fingerprint with a Verify box; stellar.expert links. One line under the fingerprint: it proves the file is unchanged since the request, and with the provider's confirmation and the payment it shows the settlement happened; it does not prove the original invoice was genuine. If `getEvents` returns events, show them as an optional timeline.
- **`/admin`.** List the verified providers; add one (check `hasTrustline` first and refuse without it); set period and max pause. Show the rulebook as a read-only table with source links. Optional if time remains: a "needs review" list from a static `reviewQueue.json`, clearly labeled "demo data", since there is no backend to store a real queue.

Pitch work

- **Outreach.** Confirm the emails went out. Add HELP for Domestic Workers, Enrich HK and Impactt as recipients and ask one question each: would you review unclear fees, would employers pay before departure, would lenders settle. Any real reply goes on a slide, quoted honestly. Never invent a partner or a funder.
- **Remediation Bond.** In the deck, replace any claim about the Impactt-Rabobank Remediation Bond with "in development" unless you find it has launched.
- **Deck, 3 minutes.** Maria's story, then the problem (the pile of bills, and a worker who borrows for costs that were never hers), then the live demo with the placement fee being refused on screen, how it works on Stellar, who pays (employers already owe these costs by law; mission funders for the rest), honest limits, what we still need to validate, and the ask. Use only evidence listed as verified in the handoff doc's Sources section.
- **One-page solution overview.** Due Fri Nov 13, 8:30 AM. You draft it by Nov 11 and Abdullah reviews. Rewrite it from v1.1 to v2.
- **Backup video.** Screen-record the whole demo script on testnet by Fri Nov 13.

Tasks

- [ ] **Now.** Confirm whether H1 (demo data) was committed, and post the status in the team chat. Confirm the outreach emails went out.

- [ ] **Today.** `providers.json`, `rulebook.json`, `people.json` update and the three sample invoices committed. Then `/creditor` and the `/funder` Requests tab on the mock (`?role=creditor`, `?role=funder`, `?role=employer`).
- [ ] **Fri Nov 6.** Funder Portfolio and `/proof/[id]` on the mock.
- [ ] **Sat Nov 7.** `/admin` (with the rulebook table). Switch to the real chain after H5 and swap in Ananya's components.
- [ ] **Sun Nov 8, H6.** Run demo steps 2 to 6 on testnet.
- [ ] **Mon Nov 9 to Wed Nov 11.** Deck v1 (Nov 10), deck v2 after advisor feedback (Nov 11), one-pager draft (Nov 11).
- [ ] **Thu Nov 12 to Sun Nov 15.** Freeze, upload the one-pager (Nov 13, 8:30 AM), backup video, two full rehearsals with the team asking judge questions.
- [ ] **Tue Nov 17 night.** Final deck done; upload by Wed Nov 18, 10 AM.

Done means: demo steps 2 to 6 run on testnet, the proof page works with no wallet, and the deck, one-pager and backup video are ready.

You wait on: Murad's H2 (for the provider addresses), Ananya's H3 (components), Abdullah's `rulebook.ts` and H5. Until then use the mock and plain shadcn components. **Others wait on you:** H1 (`providers.json`, `rulebook.json` and the sample invoices) for Ananya.

Team-wide: demo script, advisor meetings, pitch

The demo script below is the definition of done: if every step runs live on testnet, the project is finished. All people, companies and documents are fictional.

#	Who (Freighter account)	Page	What happens on screen	Owner
1	Worker (Maria)	`/worker/request`	Lists three bills: training 190 (training center), medical exam 65 (clinic), placement fee 1,380 (agency). Screen shows worker-payable, employer-owed, and a **refusal** for the placement fee with its source. Signs the two accepted bills (training first, so it is request 1)	Ananya
2	Provider (training center)	`/creditor`	Sees the 190 request, clicks **Confirm invoice**, signs	Luke
3	Provider (clinic)	`/creditor`	Same for the 65 request	Luke
4	Funder (Bayanihan Microfinance)	`/funder`	Re-checks the invoice against its fingerprint (match), chooses Loan at 3%, signs. 190 USDC moves straight to the training center. Maria owes 195.70	Luke
5	Employer	`/funder`	**Pay as employer**. 65 USDC moves to the clinic. Maria owes 0	Luke
6	Anyone	`/proof/1`, `/proof/2`	Public receipts: bill, rule and source, provider confirmation, payer, fingerprint, explorer links	Luke
7	Worker	`/worker`	Sends 400 USDC home. Result: 60 to the funder, 340 to Lorna, outstanding 195.70 to 135.70	Ananya

#	Who (Freighter account)	Page	What happens on screen	Owner
8	Worker	`/worker`	Waits one period (60 seconds). Status shows **Paused, 1 of 3**, balance unchanged. A static "without vs with Laya" card sits beside it	Ananya
9	Family	`/family`	Lorna's balance is up 340 USDC	Ananya
10	Admin (optional)	`/admin`	The rulebook table with source links, and the verified provider list	Luke

The biggest pitch moment is step 1: the placement fee being refused on screen.

Rules for the live demo. Create fresh requests live every time, because a demo "month" is 60 seconds and old loans show many missed periods. Run Murad's `demo-reset.sh` within an hour of judging. Keep Luke's backup video ready in case the venue Wi-Fi fails. Import the six accounts into Freighter on every demo laptop, set to Testnet.

Advisor meetings, Mon Nov 9 (Abdullah attends). Keesan Mahalingam 1:30-2:00 PM and Abhishek Punia 4:30-5:00 PM, both virtual. Use them to validate, not to get a technical review:

- Would a lender or mission funder fund a loan at a capped flat fee when it covers only what the worker legitimately owes?
- Would employers pay recruitment-related costs before departure if there were proof and no reimbursement dispute?
- What licensing or regulatory problems would this hit in production?
- Is there any existing funder pool, grant, or program on Stellar for something like this?

Write the answers down. Only put on a slide what an advisor actually said, and attribute it honestly.

Team tasks

- [ ] **Today, working session #1 (4-6 PM, in person).** Agree or amend the decisions listed under "Team rules". Each person says where their H-items stand. Abdullah shows the mock running the v2 flow if it is ready.

- [ ] **Every night.** A 10-minute check-in: what is done, what is blocking, which handoff is next.
- [ ] **Nov 11, working session #2.** Decide who presents what (suggested: Ananya the Maria story, Abdullah how it works, Luke the problem and the ask, Murad answers contract questions).
- [ ] **Nov 13 to Nov 15.** Two full rehearsals with the team asking the judge questions below. Time the pitch to 3 minutes.
- [ ] **Nov 16 and Nov 18.** Run demo-reset an hour before the showcase and the finals. Test the venue Wi-Fi and the backup video.

Honest answers to judge questions, and what not to claim

No step in v2 needs a party to act against its own interest. What remains is adoption risk, validation risk and challenge fit. Everyone should be able to give these answers in one or two sentences. Use an answer only if it is true on the day.

Likely question	Honest answer
Why would employers use it?	They already owe these costs by law. Laya pays before departure with proof and avoids reimbursement disputes. Adoption is unproven and is the main thing we still have to validate
What if the employer refuses to pay?	Laya cannot force it. The bill stays unpaid and marked. Hong Kong law already makes the employer liable, and the helper can claim or approach the Labour Department. Launch would go through ethical agencies that only place workers with employers who agree to pay
Who funds the loans?	A proposed market: mission lenders and co-ops accepting a capped flat fee. Employers come first (grants). Philippine senators have proposed similar remittance-repaid loans at no more than 6%: proposals, not live programs

Likely question	Honest answer
Why wouldn't workers just use their usual remittance app?	Repayment only covers remittances sent through Laya. Loans are small and short. Production would sit inside a licensed remittance partner they already use. Grants have no repayment, so no leakage
How does Laya know a bill is legal?	It does not interpret the law. It applies a published rulebook, each rule linked to its official source and kept by a legal partner. Unclear bills go to human review. A refusal never stops anyone from paying a legal fee elsewhere
Who reviews, and who is the admin?	Demo: our admin account. Production: a compliance partner mirroring public licence registers and legal aid groups that already answer "is this fee legal?". Not yet partnered
Licensed does not mean honest	True. Hong Kong regulators have fined licensed agencies for overcharging. So the register is one check among several: price caps, invoice fingerprint, human review
Isn't the legitimate amount too small to matter?	11,000 pesos is still money a family lacks before the first paycheck. The real damage is the 80,000 pesos she is pushed into borrowing, which we refuse
A recruiter can still pressure her outside Laya	Yes. Laya makes the fair path easy and the unfair one visible. It cannot stop abuse it never sees
Why Stellar?	One transaction splits a remittance between funder and family across countries for a fraction of a cent, and the public receipt can be checked without trusting us. A database could do parts of it; the shared verifiable record is the part it cannot give outsiders

Likely question	Honest answer
Is it legal?	Testnet only. Production runs as software on licensed partners (lending, payments, remittance). The 8% fee cap is Laya's own policy, not a legal claim
Has this been done?	Employers repay recruitment fees at scale (Impactt reports over US$111 million to 86,000 workers) but after the fact and with estimates. We found no product that pays providers before departure, sorts who owes each cost, and publishes the receipt
Does this fit "loans that fit real income"?	We finance what the worker legitimately owes with repayment that flexes with her remittances, and we stop the predatory loans that fill the gap today. Keep the loan step prominent in the demo. We are in problem space 1, which is what matters

You may state as fact (read from official or major sources)

- The Hong Kong employer is responsible for the helper's document-preparation expenses and her pre-entry medical exam, and a helper whose employer will not reimburse can bring a civil claim or ask the Labour Department for help.
- Hong Kong agencies may charge a helper at most 10% of her first month's wages, only after successful placement. The Philippines has a zero placement fee policy for domestic workers, and agencies may still charge medical and training fees.
- Migrasia's 2022 survey of 961 randomly selected OFWs found 80.5% went into debt for their placement, about a third took on debt larger than annual household income, and two in five felt pressured to use a specific clinic, training center or lender.
- Complaint documents describe lender rates of 61% to 578% a year (ICIJ, reported as alleged), post-dated or blank check pressure, and a jailed worker (PCIJ). A training fee of about 11,000 pesos was reported charged at 80,000 to 90,000 pesos.
- Impactt reports over US$111 million repaid to 86,000+ workers across 210 companies. A Transparentem-reported Malaysian fund paid $1,067,684 of $1.1 million promised. The Impactt-Rabobank Remediation Bond is described as in development.

Do not state as fact

- Any live funder, partner, advisor endorsement or program. None exists yet.
- That the 11,000-peso training figure is an official price cap. It is one reported figure. Our rulebook caps (100 USDC for the medical exam, 250 USDC for training) are illustrative demo values.
- The "borrowed 50,000 pesos, repaid 76,800" figure from the original plan. It was not found.
- The 2012 union survey (HK$15,378 average agency fee; 68% borrowed). It is old and from an advocacy group.
- That the DMW "employer bears recruitment cost" rule is official. It comes from a secondary legal commentary.
- That training centers and clinics have official registers like the agency register. Not checked.
- Hong Kong's 380,000 foreign domestic workers and about 1,400 agencies. Older press figures.
- That the fingerprint proves a document is genuine. It proves it is unchanged.

Sources

- HK Labour Department: FDH guide
- HK Government: employer reimbursement of FDH costs
- HK Employment Agencies Administration: what agencies may charge
- SCMP: zero placement fee policy
- PCIJ: Migrasia survey and OFW debt
- PCIJ: harassment and jail over loans
- ICIJ: lenders and labor agents
- Transparentem: Malaysia reimbursement
- Impactt: repayment results
- Impactt: Remediation Bond
- Philippine Senate: proposed OFW credit-assistance loan

These were read from the cited pages in the project's earlier chat, as recorded in the handoff doc. Nothing here is legal advice; the rulebook needs a legal partner before real use.